

# Archivo 1 — Análisis del Proyecto

## 1. Presentación del proyecto

### 1.1 Objetivo

zz-js01 es una aplicación web de aprendizaje que implementa un **CRUD completo de productos**. La sigla CRUD reúne las cuatro operaciones básicas sobre datos: **Create** (crear), **Read** (listar), **Update** (editar) y **Delete** (borrar). Quien usa la aplicación puede cargar un producto con un formulario (nombre y precio), verlo en una tabla, editarlo y borrarlo; los datos quedan guardados en una base de datos y sobreviven al cierre del navegador y del servidor.

El proyecto está comentado línea por línea con la intención de leerse de arriba hacia abajo, y este paquete pedagógico acompaña esa lectura.

### 1.2 Tecnologías empleadas (stack)

El conjunto de tecnologías de un proyecto se conoce como **stack**. El stack de zz-js01 es:

| Capa                 | Tecnología                                                         | Rol en el proyecto                                                                                                                                          |
|----------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Entorno de ejecución | Node.js (se requiere versión 22 o superior)                        | Ejecuta JavaScript del lado del servidor.                                                                                                                   |
| Servidor web         | Express 5 (paquete npm <code>express</code> )                      | Framework ( <b>framework</b> : biblioteca que organiza la aplicación y resuelve las tareas repetitivas de un servidor web) que atiende las peticiones HTTP. |
| Base de datos        | SQLite mediante el paquete <code>better-sqlite3</code>             | Guarda los productos en un único archivo del disco.                                                                                                         |
| Frontend             | HTML + CSS + JavaScript puro, sin frameworks                       | Formulario y tabla que el navegador muestra; se comunica con el servidor mediante <code>fetch()</code> .                                                    |
| Pruebas              | <code>node:test</code> (incluido en Node) + <code>supertest</code> | Pruebas automatizadas de la API ( <b>API</b> : interfaz que un programa expone para que otros programas la consuman; aquí, un conjunto de rutas HTTP).      |

El proyecto usa módulos CommonJS ( `require` / `module.exports` ), como declara el campo `"type": "commonjs"` de `package.json`.

### 1.3 Qué puede hacer quien usa la aplicación

Desde el navegador, en `http://localhost:3000/`, se puede:

- **Crear** un producto completando el formulario (nombre y precio) y presionando Guardar.
- **Listar** los productos: la tabla muestra todos los guardados, con su id, nombre y precio.
- **Editar** un producto con el botón Editar de cada fila: sus valores se cargan en el formulario y Guardar pasa a decir "Guardar cambios".
- **Borrar** un producto con el botón Borrar de cada fila, previa confirmación.

Cada una de esas acciones de la interfaz corresponde a una operación HTTP sobre la API de productos, descrita en las secciones siguientes.

## 2. Arquitectura general

### 2.1 Flujo de una petición

La arquitectura del proyecto es la clásica de una aplicación web de tres capas: navegador, servidor y base de datos. El recorrido de una petición es el siguiente:

```

Navegador (public/index.html)
  | 1) petición HTTP con fetch(): GET, POST, PUT o DELETE a /api/productos
  v
Express (src/app.js: express.json -> express.static -> router)
  | 2) la ruta correspondiente en src/routes/productos.js
  | 3) validación con src/validators/productos.js (solo POST y PUT)
  v
better-sqlite3 (src/db.js)
  | 4) consulta SQL con prepared statements
  v
Archivo SQLite (mi_base_de_datos.db)
  | 5) resultado: filas, changes o lastInsertRowid
  v
Express responde HTTP (200 / 201 / 204 / 400 / 404)
  | 6) el navegador recibe la respuesta y actualiza la página
  v
Navegador (la tabla se vuelve a dibujar con cargarProductos())

```

En síntesis: **Navegador** → **HTTP** → **Express** → **better-sqlite3** → **archivo SQLite**, y de vuelta por el mismo camino con la respuesta.

### 2.2 Estructura de carpetas y archivos

| Archivo / carpeta          | Responsabilidad                                                                                       |
|----------------------------|-------------------------------------------------------------------------------------------------------|
| <code>src/app.js</code>    | Arma la aplicación Express: middlewares, montaje del router.<br>No escucha puertos.                   |
| <code>src/server.js</code> | Único punto de arranque: pregunta si poblar la base<br>(instalación limpia) y pone la app a escuchar. |

| Archivo / carpeta                        | Responsabilidad                                                                                                   |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>src/db.js</code>                   | Abre la conexión SQLite, crea la tabla <code>productos</code> y exporta <code>{ db, poblarDesdeArchivo }</code> . |
| <code>src/routes/productos.js</code>     | Define las cuatro rutas de la API: GET, POST, PUT y DELETE.                                                       |
| <code>src/validators/productos.js</code> | Reglas de validación de un producto; devuelve todos los errores encontrados.                                      |
| <code>public/index.html</code>           | Frontend completo: formulario, tabla y todo el JavaScript del navegador.                                          |
| <code>test/api.test.js</code>            | 12 pruebas automatizadas de la API (CRUD y validaciones).                                                         |
| <code>test/poblar.test.js</code>         | 1 prueba automatizada del seed de datos de ejemplo.                                                               |
| <code>poblar_base.sql</code>             | Script SQL con los 5 productos de ejemplo.                                                                        |
| <code>package.json</code>                | Declara dependencias y scripts ( <code>start</code> , <code>test</code> ).                                        |
| <code>mi_base_de_datos.db</code>         | La base SQLite; se crea sola en el primer arranque y no se versiona.                                              |

## 2.3 Por qué el frontend vive en public/

El archivo `public/index.html` no se abre como archivo local ( `file:///...` ): se sirve por HTTP mediante el middleware `express.static` . La razón técnica es que el frontend usa **rutas relativas** ( `"/api/productos"` ): el navegador las completa con el origen de la página, que solo existe si la página fue servida por un servidor. La razón de seguridad es complementaria: al apuntar `express.static` a `public/` — y no a la raíz del proyecto — quedan expuestos por HTTP únicamente los archivos de la interfaz; la base de datos, el código del servidor y el `package.json` permanecen inaccesibles desde el navegador. El propio comentario de `src/app.js` lo declara:

```
// Middleware de archivos estáticos: Express busca el archivo pedido
// dentro de public/ y lo sirve tal cual; si no existe, sigue hacia
// las rutas de la API (GET / sirve index.html automáticamente).
// Se apunta a public/ -y no a la raíz del proyecto- para no exponer
// por HTTP la base de datos, este código ni el package.json.
app.use(express.static(path.join(__dirname, "..", "public")));
```

## 2.4 Por qué app.js no escucha puertos

La aplicación Express se arma en `src/app.js` pero se exporta **sin** llamar a `listen()` ; el arranque vive únicamente en `src/server.js` . Esta separación permite que las pruebas requieran la app directamente ( `test/api.test.js` con `supertest` ) y le envíen peticiones HTTP sin ocupar un puerto real ni interferir con un servidor en ejecución. El cierre de `src/app.js` lo explicita:

```
// Se exporta la app SIN escuchar puertos: así los tests pueden
// requerirla (test/api.test.js con supertest) y src/server.js es
```

```
// el único archivo que la arranca con app.listen().
module.exports = app;
```

## 3. Desglose técnico por módulos

### 3.1 src/app.js — la aplicación Express

**Responsabilidad:** crear la app, registrar los middlewares en orden y montar el router de productos. Un **middleware** es una función por la que Express hace pasar cada petición, en el orden en que se la registra, hasta que alguna responde.

El primer middleware interpreta el cuerpo JSON de las peticiones POST y PUT:

```
// Middleware: interpreta el body JSON de los pedidos POST/PUT y lo
// deja disponible en req.body como objeto JavaScript.
// Sin esta línea, req.body llegaría undefined.
app.use(express.json());
```

El segundo sirve el frontend (sección 2.3) y el tercero monta el router bajo el prefijo `/api/productos`:

```
app.use("/api/productos", productosRouter);
```

#### Decisiones de diseño notables:

- `express.json()` se registra **antes** de las rutas: si se registrara después, las rutas leerían `req.body` antes de que exista.
- El orden de la cadena importa: `express.json()` → `express.static(...)` → router.
- La app se exporta sin escuchar (sección 2.4).

### 3.2 src/server.js — el arranque del servidor

**Responsabilidad:** ser el único punto de entrada. Arma la app y la deja escuchando en un puerto. Toda la lógica vive en `src/`; este archivo solo orquesta el arranque.

La novedad de la etapa 8 es la pregunta de instalación limpia: si el archivo de la base todavía no existe, el servidor pregunta por terminal si poblarlo con los datos de ejemplo de `poblar_base.sql`. La pregunta vive **solo** aquí: `app.js` y las pruebas jamás piden nada por terminal.

```
let poblar = false;
if (!fs.existsSync(DB_PATH)) poblar = await preguntarPoblar();

// Aquí sí: require("../db") crea el archivo (si faltaba) y asegura
// la tabla (CREATE TABLE IF NOT EXISTS); require("../app") arma la
// aplicación Express. CommonJS cachea los require: es la MISMA
```

```
// conexión que usan las rutas.
const { db } = require("./db");
const app = require("./app");
```

El orden es deliberado y está comentado en el propio archivo: `require("./db")` crea el archivo de la base apenas se ejecuta, por lo que la pregunta debe hacerse **antes** de ese require; de lo contrario, en una instalación limpia el archivo ya existiría cuando se llegara a preguntar, y la pregunta nunca se haría.

La pregunta en sí rechaza los contextos sin terminal interactiva y aplica la convención `[s/N]` (la mayúscula indica la respuesta por defecto: solo Enter significa **no** poblar):

```
if (!process.stdin.isTTY) {
  console.log(
    "Instalación limpia detectada. Sin terminal interactiva: se crea la base vacía.",
  );
  return Promise.resolve(false);
}
```

```
.then((respuesta) => {
  const limpio = respuesta.trim().toLowerCase(); // " S\n" → "s"
  // Aceptación explícita: solo estas cuatro respuestas pueblan.
  // Cualquier otra cosa (Enter vacío, "n", "no", un grito) no.
  return ["s", "si", "y", "yes"].includes(limpio);
})
.finally(() => rl.close()); // cierra la interfaz SIEMPRE, acepte o rechace
```

Dos decisiones más completan el arranque:

- El seed es **opcional**: si `poblar_base.sql` faltara o fallara, el servidor informa el error y arranca igual, con la base vacía; el CRUD no depende de los datos de ejemplo.
- El puerto se toma del entorno: `const PORT = process.env.PORT || 3000;`, lo que permite cambiarlo desde la terminal ( `PORT=3100 node src/server.js` ) sin tocar el código.

```
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Servidor corriendo en http://localhost:${PORT}`);
});
```

### 3.3 src/db.js — la conexión SQLite

**Responsabilidad:** abrir la base, asegurar la tabla `productos` y exportar la conexión junto con la función de seed.

La ruta del archivo de base se puede cambiar con la variable de entorno `DB_PATH` (las pruebas la apuntan a un archivo temporal para no tocar la base real):

```
const DB_PATH = process.env.DB_PATH || "mi_base_de_datos.db";
const db = new Database(DB_PATH);
```

**better-sqlite3** es un driver **síncrono**: cada consulta se ejecuta y devuelve su resultado de inmediato, sin promesas ni callbacks. No existe un servidor de base de datos separado: toda la base vive en el único archivo **.db**.

La tabla se asegura una sola vez al arrancar, con **IF NOT EXISTS**, lo que permite reiniciar el servidor sin perder datos y sin errores si la tabla ya existe:

```
db.exec(`
  CREATE TABLE IF NOT EXISTS productos (
    id INTEGER PRIMARY KEY AUTOINCREMENT, -- id autogenerado: 1, 2, 3...
    nombre TEXT NOT NULL,                 -- texto obligatorio
    precio REAL NOT NULL                   -- número con decimales
  )
`);
```

El módulo exporta un objeto con dos miembros; quien lo requiera debe desestructurarlo:

```
module.exports = { db, poblarDesdeArchivo };
```

La función de seed lee un archivo **.sql** completo y lo ejecuta de una sola vez con **db.exec()**, que admite múltiples sentencias seguidas:

```
function poblarDesdeArchivo(rutaSql) {
  const sql = fs.readFileSync(rutaSql, "utf8"); // se lee TODO el archivo a un
  string
  db.exec(sql); // y se ejecuta entero de una (exec sí soporta múltiples sentencias)
}
```

**Decisión de diseño notable:** la tabla **productos** no tiene restricción **UNIQUE**, de modo que ejecutar el seed dos veces insertaría los productos duplicados. La función no verifica nada al respecto: confía en que se la invoque una sola vez, y en la práctica solo lo hace **src/server.js**, únicamente cuando la base era nueva. La prueba **test/poblar.test.js** protege este contrato al exigir exactamente 5 filas.

### 3.4 src/routes/productos.js — las rutas de la API

**Responsabilidad:** definir y responder las cuatro operaciones del CRUD. El archivo crea un router (**express.Router()**), una mini-aplicación de Express que **src/app.js** monta bajo **/api/productos**; una ruta definida aquí como **/** responde en **/api/productos** y una definida como **/:id** responde en **/api/productos/:id**.

Las conexiones se importan desestructurando los módulos:

```
const { db } = require("../db");
const { validarProducto } = require("../validators/productos");

const router = express.Router();
```

**GET — listar todos (200).** `prepare()` compila la consulta una sola vez; `.all()` devuelve todas las filas como array de objetos, y `res.json()` convierte y responde:

```
router.get("/", (_req, res) => {
  // prepare() compila la consulta una sola vez (eficiente si se repite).
  // El guion bajo en _req indica: "Express me exige este parámetro
  // (el pedido), pero no lo uso en esta ruta".
  const productos = db.prepare("SELECT * FROM productos").all();
  res.json(productos); // res.json(): convierte a JSON y responde
});
```

**POST — crear (201 o 400).** Primero se valida; solo con datos válidos se inserta. El nombre se normaliza con `trim()` (recorte de espacios de los extremos) y la respuesta 201 incluye el id autogenerado:

```
// Ya validado: trim() normaliza el nombre (sin espacios sobrantes).
const nombre = req.body.nombre.trim();
const precio = req.body.precio;
const insert = db.prepare(
  "INSERT INTO productos (nombre, precio) VALUES (?, ?)",
);
// run() ejecuta el INSERT; lastInsertRowid trae el id generado.
const resultado = insert.run(nombre, precio);
// 201 Created: convención REST para "se creó un recurso nuevo".
res.status(201).json({ id: resultado.lastInsertRowid, nombre, precio });
```

El patrón de validación previa es idéntico en POST y en PUT (una sola función de validación, dos rutas):

```
router.put("/:id", (req, res) => {
  // Mismas reglas para crear y editar: una sola función, dos rutas.
  const errores = validarProducto(req.body);
  if (errores.length > 0) {
    return res.status(400).json({ errores });
  }
  const nombre = req.body.nombre.trim(); // ya validado y normalizado
  const precio = req.body.precio;
```

**PUT — reemplazar (200, 400 o 404).** El `WHERE id = ?` limita la actualización a la fila indicada; sin él se actualizarían todas. Si `changes` es 0, ninguna fila coincidió con el id: no existía y se responde 404:

```
// UPDATE ... WHERE id = ?: cambia SOLO las filas que matcheen el id.
// Sin WHERE se actualizarían TODAS las filas: error clásico y caro.
const actualizar = db.prepare(
  "UPDATE productos SET nombre = ?, precio = ? WHERE id = ?",
);
// Tres valores, tres "?": en el mismo orden que aparecen en el SQL.
const resultado = actualizar.run(nombre, precio, req.params.id);
// Mismo patrón changes que en DELETE: 0 significa "ese id no existe".
if (resultado.changes === 0) {
  return res.status(404).json({ error: "Producto no encontrado" });
}
// Devolvemos cómo quedó el producto. Number(): req.params.id es string,
// lo convertimos para que el JSON muestre el id como número.
res.json({ id: Number(req.params.id), nombre, precio });
```

**DELETE — borrar (204 o 404).** Mismo patrón de `changes` ; un borrado exitoso no tiene contenido que devolver, por eso responde 204 con `.end()` :

```
router.delete("/:id", (req, res) => {
  const borrar = db.prepare("DELETE FROM productos WHERE id = ?");
  // run() devuelve "changes": cuántas filas fueron borradas de verdad.
  const resultado = borrar.run(req.params.id);
  // Si changes es 0, no existía ningún producto con ese id. Responder
  // 404 (Not Found) es más honesto que un 204 silencioso: quien llamó
  // pidió borrar algo que no estaba.
  if (resultado.changes === 0) {
    return res.status(404).json({ error: "Producto no encontrado" });
  }
  // 204 No Content: "salió bien y no tengo nada que devolver". Es la
  // convención REST para borrados exitosos. .end() cierra la respuesta
  // sin body.
  res.status(204).end();
});
```

### Decisiones de diseño notables:

- Las cuatro consultas usan **prepared statements** con marcadores `?` : los valores viajan separados del SQL, de modo que un dato del formulario no puede alterar la consulta (**inyección SQL**: técnica de ataque que consiste en introducir SQL malicioso a través de los datos; ver Archivo 5, unidad 5.4).
- `req.params.id` llega siempre como **string**; SQLite lo compara correctamente contra el id numérico, y PUT lo convierte con `Number()` solo para presentarlo en la respuesta.
- La validación se ejecuta **siempre antes** de tocar la base: el `required` del formulario ayuda al usuario, pero el servidor no confía en nadie, porque un cliente como `curl` puede saltarse el HTML completo.

## 3.5 src/validators/productos.js — la validación

**Responsabilidad:** verificar el body de un producto contra las reglas de negocio y devolver **todos** los problemas encontrados en un array (array vacío = producto válido). Así quien consume la API puede corregir todo de una vez, en lugar de descubrir los errores de a uno por intento.

```
function validarProducto(body) {
  // Si el pedido llegó sin body, req.body puede ser undefined:
  // lo reemplazamos por un objeto vacío para no reventar acá
  // (un body vacío simplemente no valida nada).
  const datos = body || {};
  const errores = [];

  // --- nombre ---
  // typeof revisa el TIPO primero: si llega un número o un objeto,
  // no tiene sentido medir el largo. trim() saca espacios de los
  // extremos, así "  " cuenta como vacío.
  if (typeof datos.nombre !== "string" || datos.nombre.trim() === "") {
    errores.push("El nombre es obligatorio y debe ser texto.");
  } else if (datos.nombre.trim().length > 100) {
    errores.push("El nombre no puede superar los 100 caracteres.");
  }

  // --- precio ---
  // Number.isFinite acepta 10 y 10.5; rechaza NaN, "10" (string),
  // Infinity y objetos. TRAMPA CLÁSICA: no usar !precio, porque
  // además de NaN rechazaría cualquier valor falsy sin distinguir.
  if (!Number.isFinite(datos.precio)) {
    errores.push("El precio es obligatorio y debe ser un número.");
  } else if (datos.precio < 0.01) {
    // Regla de negocio: nada gratis, el mínimo es 0.01.
    errores.push("El precio mínimo permitido es 0.01.");
  }

  return errores;
}
```

### Decisiones de diseño notables:

- El guard `body || {}` evita el error de leer propiedades de `undefined` cuando la petición llega sin cuerpo.
- Para el precio se usa `Number.isFinite` y no `!precio`: la negación rechazaría sin distinguir cualquier valor *falsy* (entre ellos el `0` legítimo de otros contextos), mientras que `Number.isFinite` acepta exactamente los números finitos y rechaza `NaN`, textos como `"10"`, `Infinity` y objetos. Sobre esta trampa ver Archivo 5, unidad 6.1.
- Las reglas vigentes: nombre — texto obligatorio, con espacios recortados, de máximo 100 caracteres; precio — número finito, mínimo 0.01 (no se aceptan 0 ni negativos).

## 3.6 public/index.html — el frontend

**Responsabilidad:** toda la interfaz: formulario de alta/edición, tabla de listado y la totalidad del JavaScript del navegador, sin frameworks. Se comunica con el backend únicamente por HTTP con `fetch()` y rutas relativas, por lo que debe abrirse vía `http://localhost:3000/` y no como archivo local.

El formulario declara sus dos validaciones visuales del lado del navegador:

```
<form id="formulario-producto">
  <!-- required: el navegador valida solo que no esté vacío -->
  <input type="text" id="nombre" placeholder="Nombre del producto" required>
  <!-- step 0.01: admite decimales (precios con centavos) -->
  <input type="number" step="0.01" id="precio" placeholder="Precio" required>
```

Las referencias al DOM se buscan una vez y se guardan en constantes:

```
// Referencias al DOM (la página). Se buscan una vez y se guardan
// en constantes: más legible y rápido que buscarlas en cada uso.
const formulario = document.getElementById('formulario-producto');
const tbody = document.getElementById('lista-productos');
const botonGuardar = document.getElementById('boton-guardar');
const botonCancelar = document.getElementById('boton-cancelar');
```

Dos variables de estado gobiernan todo el comportamiento: `idEnEdicion` decide si el submit envía POST (crear) o PUT (editar), y `productosActuales` guarda la última lista traída del backend para rellenar el formulario al editar sin pedidos extra:

```
// Estado del formulario. idEnEdicion: null = creando un producto
// nuevo; un número = editando ese id. Esta única variable decide
// si el submit manda POST (crear) o PUT (actualizar).
let idEnEdicion = null;
// Última lista traída del backend: editarProducto() la usa para
// rellenar el formulario sin volver a pedir ese producto.
let productosActuales = [];
```

El manejador del submit cancela la recarga de página, convierte el precio a número y encadena guardar y refrescar:

```
formulario.addEventListener('submit', async (evento) => {
  // preventDefault() cancela el comportamiento por defecto del
  // submit (recargar la página). Sin esto, la recarga cortaría
  // nuestro JavaScript antes de llegar a enviar el pedido.
  evento.preventDefault();

  // .value siempre devuelve texto; para el precio hace falta
  // parseFloat() y así trabajar con un número de verdad.
  const nombre = document.getElementById('nombre').value;
  const precio = parseFloat(document.getElementById('precio').value);
```

```
// await: espera el resultado del guardado (POST o PUT, según
// el modo) antes de continuar. Si falla, guardado es false y
// el formulario NO se limpia: no se pierde lo escrito por un
// error del servidor.
const guardado = await guardarProducto(nombre, precio);
if (guardado) {
  // salirDeEdicion() limpia el formulario Y devuelve el estado a
  // "creando" (texto del botón incluido). Sirve para ambos caminos.
  salirDeEdicion();
  // Ciclo POST -> GET: recién ahora que el producto está en la
  // base, se pide la lista actualizada. El await garantiza ese
  // orden (sin él, ambos correrían en paralelo).
  await cargarProductos();
}
});
```

`guardarProducto` arma la URL según el modo y envía el body serializado como JSON; además revisa `respuesta.ok` a mano, porque `fetch` no lanza excepción ante códigos de error HTTP:

```
// ¿Creando o editando? Misma ruta base; cambia el método y, al
// editar, el id viaja en la URL: PUT /api/productos/7.
const creando = idEnEdicion === null;
const url = creando ? '/api/productos' : `/api/productos/${idEnEdicion}`;
const respuesta = await fetch(url, {
  method: creando ? 'POST' : 'PUT',
  // El backend necesita saber que el body es JSON
  headers: { 'Content-Type': 'application/json' },
  // fetch solo envía texto: el objeto debe serializarse
  body: JSON.stringify({ nombre, precio })
});
```

```
if (!respuesta.ok) {
  console.error('El servidor rechazó el guardado:', respuesta.status);
  return false;
}
const datos = await respuesta.json(); // { id, nombre, precio }
```

La edición se apoya en `find()` sobre la lista ya cargada:

```
function editarProducto(id) {
  // find() devuelve el primer elemento que cumple la condición
  // (o undefined si no está): se busca en la lista ya cargada.
  const producto = productosActuales.find((p) => p.id === id);
  if (!producto) return;
  idEnEdicion = id;
  document.getElementById('nombre').value = producto.nombre;
  document.getElementById('precio').value = producto.precio;
  botonGuardar.textContent = 'Guardar cambios';
}
```

```

    botonCancelar.hidden = false; // ahora sí se ve el botón Cancelar
  }

```

El borrado pide confirmación nativa antes de enviar el DELETE:

```

async function borrarProducto(id) {
  // confirm() muestra un diálogo nativo del navegador y devuelve
  // true/false según lo que responda el usuario. Es la barrera mínima
  // contra un clic accidental sobre "Borrar".
  if (!confirm(`¿Borrar el producto #${id}?`)) return;
  try {
    const respuesta = await fetch(`/api/productos/${id}`, {
      method: 'DELETE'
    });
  }
}

```

El listado redibuja la tabla completa construyendo los nodos con `createElement` y llenando las celdas con `textContent`, que inserta **texto plano** y nunca interpreta HTML. Esa elección es una defensa contra **XSS** (*Cross-Site Scripting*: ataque que consiste en inyectar código que el navegador ejecutaría; con `innerHTML`, un nombre como `<script>...` se ejecutaría). Vaciar el contenedor con `tbody.innerHTML = ''` sí es seguro, porque no interpola ningún dato:

```

// Vaciar y redibujar todo. Con muchos registros se optimizaría
// actualizando solo lo que cambió.
tbody.innerHTML = '';
for (const producto of productos) {
  const fila = document.createElement('tr');
  // toFixed(2): formato monetario, siempre 2 decimales
  const valores = [producto.id, producto.nombre, `$
${Number(producto.precio).toFixed(2)}`];
  for (const valor of valores) {
    const celda = document.createElement('td');
    celda.textContent = valor;
    fila.appendChild(celda);
  }
}

```

Cada fila recibe sus botones, y la función flecha del botón Borrar actúa como clausura que guarda el id de **esa** fila:

```

const botonBorrar = document.createElement('button');
botonBorrar.textContent = 'Borrar';
// La arrow function hace de "clausura": guarda el producto.id de
// ESTA fila y solo se ejecuta cuando se hace clic en ese botón.
botonBorrar.addEventListener('click', () => borrarProducto(producto.id));
celdaAccion.appendChild(botonBorrar);
fila.appendChild(celdaAccion);
tbody.appendChild(fila);

```

**Decisiones de diseño notables:**

- Un único estado ( `idEnEdicion` ) distingue crear de editar: el mismo formulario y el mismo submit sirven para POST y PUT.
- El ciclo de mutación es siempre **mutar y después refrescar** (POST/PUT/DELETE y recién luego `cargarProductos()` ), con `await` garantizando ese orden.
- Si un guardado falla, el formulario no se limpia: no se pierde lo escrito por un error del servidor.
- El redibujado completo es simple y suficiente para la cantidad de datos de este proyecto.

### 3.7 test/api.test.js — las pruebas de la API

**Responsabilidad:** 12 pruebas automatizadas con el corredor nativo `node:test` , la librería de aserciones `node:assert` y `supertest` , que dispara peticiones HTTP contra la app directamente, sin levantar el servidor en un puerto real.

El detalle clave del archivo es el **aislamiento**: `DB_PATH` se define **antes** de cualquier `require` de `src/` , porque `src/db.js` abre la base apenas se lo requiere y su ruta sale de esa variable:

```
// Va PRIMERO, antes de requerir la app: DB_PATH apunta a un archivo
// temporal con nombre único (Date.now() evita colisiones entre
// corridas del mismo test o de dos terminales a la vez).
process.env.DB_PATH = path.join(os.tmpdir(), `zz-js01-test-${Date.now()}.db`);
```

```
const { test, after } = require("node:test");
const assert = require("node:assert");
const request = require("supertest");
const app = require("../src/app");
```

Al finalizar todas las pruebas se cierra la conexión y se borra la base temporal (en Windows no se puede borrar un archivo abierto, por eso `db.close()` precede al borrado):

```
after(() => {
  db.close();
  fs.rmSync(process.env.DB_PATH, { force: true });
});
```

Las pruebas se organizan en dos grupos. Los casos 1 a 6 forman la **cadena feliz** del CRUD: comparten un producto real cuyo id se obtiene del POST y se va pasando de prueba en prueba (POST 201 → GET 200 → PUT 200 → PUT inexistente 404 → DELETE 204 → DELETE repetido 404):

```
test("POST con datos válidos responde 201 y el producto con id", async () => {
  const res = await request(app)
    .post("/api/productos")
    .send({ nombre: "Teclado mecánico", precio: 25.5 });
  assert.strictEqual(res.status, 201);
  assert.ok(res.body.id, "el body debe traer un id generado");
  assert.strictEqual(res.body.nombre, "Teclado mecánico");
  assert.strictEqual(res.body.precio, 25.5);
});
```

```
idCreado = res.body.id; // los siguientes tests de la cadena lo usan
});
```

```
test("PUT sobre un id inexistente responde 404", async () => {
  const res = await request(app)
    .put("/api/productos/999999")
    .send({ nombre: "Fantasma", precio: 10 });
  assert.strictEqual(res.status, 404);
  assert.deepStrictEqual(res.body, { error: "Producto no encontrado" });
});
```

Los casos 7 a 12 cubren la validación: body vacío con exactamente 2 errores, nombre de solo espacios, precio texto, precio 0, nombre de 101 caracteres, y la acumulación de ambos errores juntos en un PUT inválido:

```
test("PUT con nombre vacío y precio -1 responde 400 con ambos errores", async () =>
{
  const res = await request(app)
    .put("/api/productos/999999")
    .send({ nombre: "", precio: -1 });
  assert.strictEqual(res.status, 400);
  assert.deepStrictEqual(res.body.errores, [
    "El nombre es obligatorio y debe ser texto.",
    "El precio mínimo permitido es 0.01.",
  ]);
});
```

**Decisión de diseño notable:** las pruebas aprovechan la caché de módulos de CommonJS: el `../src/db` requerido por el test y el requerido dentro de las rutas son **el mismo** objeto, de modo que la prueba cierra exactamente la conexión que la app abrió.

### 3.8 test/poblar.test.js — la prueba del seed

**Responsabilidad:** verificar que `poblarDesdeArchivo()` lee `poblar_base.sql` y deja exactamente los 5 productos de ejemplo en la tabla. Utiliza su propio archivo temporal (con prefijo distinto al de `api.test.js`) y corre en su propio proceso, de modo que no interfiere con las demás pruebas.

```
test("poblarDesdeArchivo carga exactamente los 5 productos de poblar_base.sql", ()
=> {
  // La ruta del seed se arma igual que en src/server.js: este test
  // vive en test/, así que ".." sube a la raíz del proyecto.
  poblarDesdeArchivo(path.join(__dirname, "..", "poblar_base.sql"));

  // SELECT COUNT(*) devuelve una fila con una columna; .get() la trae
  // como objeto { c: <número> } gracias al alias AS c.
  const { c } = db.prepare("SELECT COUNT(*) AS c FROM productos").get();
  assert.strictEqual(c, 5, "el seed debe insertar exactamente 5 productos");
});
```

```
// Verificación de contenido: un producto del seed debe estar, con
// su precio exacto, para confirmar que vino de poblar_base.sql.
const teclado = db
  .prepare("SELECT nombre, precio FROM productos WHERE nombre = ?")
  .get("Teclado Mecánico");
assert.ok(teclado, "el seed debe insertar 'Teclado Mecánico'");
assert.strictEqual(teclado.precio, 45.99);
});
```

**Decisión de diseño notable:** la exigencia de exactamente 5 filas delata cualquier duplicación: como la tabla no tiene `UNIQUE`, un doble seed produciría 10 filas y esta prueba fallaría.

## 4. Flujo de datos de una operación completa

### 4.1 Crear un producto (POST)

1. La persona usuaria completa nombre y precio y presiona Guardar ( `type="submit"` ).
2. El navegador dispara el evento `submit`; el manejador de `public/index.html` ejecuta `evento.preventDefault()` para evitar la recarga de la página.
3. Los valores se leen con `.value`; el precio se convierte con `parseFloat()` porque `.value` entrega texto.
4. Se llama a `guardarProducto(nombre, precio)` con `await`. Como `idEnEdicion` es `null`, `creando` es verdadero: la URL es `/api/productos` y el método, POST.
5. `fetch` envía la petición con el encabezado `Content-Type: application/json` y el body producido por `JSON.stringify({ nombre, precio })`.
6. En el servidor, el middleware `express.json()` convierte ese texto en el objeto `req.body`.
7. La ruta POST de `src/routes/productos.js` invoca `validarProducto(req.body)`. Si hay errores, responde **400** con la lista completa; el frontend detecta `!respuesta.ok`, registra el error en consola y devuelve `false`, de modo que el formulario **no** se limpia.
8. Con datos válidos: se recorta el nombre ( `trim()` ), se ejecuta el prepared statement `INSERT INTO productos (nombre, precio) VALUES (?, ?)` y se responde **201** con `{ id, nombre, precio }`, donde el id proviene de `lastInsertRowid`.
9. De vuelta en el navegador, `respuesta.ok` es verdadero; se registra el id en consola y `guardarProducto` devuelve `true`.
10. Se ejecuta `salirDeEdicion()` (limpia el formulario y restaura el modo creación) y, con `await`, `cargarProductos()`.
11. `cargarProductos()` hace **GET** `/api/productos`, recibe el array JSON, guarda la lista en `productosActuales` y redibuja la tabla con `createElement` + `textContent`. El producto nuevo aparece en la tabla.

### 4.2 Editar un producto (PUT)

1. La persona usuaria presiona el botón Editar de una fila.
2. La clausura del botón llama a `editarProducto(producto.id)`.

3. `find()` localiza el producto en `productosActuales`; sus valores se cargan en el formulario, `idEnEdicion` toma el valor del id, el botón pasa a decir "Guardar cambios" y aparece el botón Cancelar.
4. Al confirmar el formulario, el submit sigue el mismo camino que en la creación (pasos 2 y 3 de la sección 4.1), pero ahora `creando` es falso: la URL es ``/api/productos/${id}`` y el método, PUT.
5. En el servidor, la ruta PUT valida igual que POST; con datos válidos ejecuta `UPDATE productos SET nombre = ?, precio = ? WHERE id = ?` pasando tres valores en orden.
6. Si `resultado.changes` es 0, el id no existe y se responde **404**; si afectó una fila, se responde **200** con el producto resultante ( `Number(req.params.id)` convierte el id a número para el JSON).
7. En el navegador, tras un `respuesta.ok` verdadero se ejecuta `salirDeEdicion()` y `cargarProductos()`: la tabla muestra los datos actualizados.

### 4.3 Borrar un producto (DELETE)

1. La persona usuaria presiona el botón Borrar de una fila.
2. La clausura del botón llama a `borrarProducto(producto.id)`; `confirm()` muestra el diálogo nativo `¿Borrar el producto #id?` y un rechazo aborta la operación.
3. `fetch` envía **DELETE** a ``/api/productos/${id}``: el id viaja en la URL, no hay body.
4. En el servidor, la ruta DELETE ejecuta el prepared statement `DELETE FROM productos WHERE id = ?`.
5. Si `changes` es 0, se responde **404**; en caso contrario, **204** sin contenido ( `.end()` ).
6. En el navegador, tras un `respuesta.ok` verdadero: si justo se estaba editando el producto borrado, `salirDeEdicion()` devuelve el formulario al modo creación; luego `cargarProductos()` redibuja la tabla sin la fila eliminada.

## 5. Códigos de estado HTTP empleados

Un **código de estado** es el número que toda respuesta HTTP incluye para resumir su resultado. La familia 2xx comunica éxito; la 4xx, un error atribuible al cliente. El proyecto emplea exactamente cinco:

| Código | Nombre     | Significado                                                         | Cuándo se devuelve en este proyecto                                                                                 |
|--------|------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 200    | OK         | La solicitud se atendió con éxito y la respuesta incluye contenido. | <code>GET /api/productos</code> (la lista) y <code>PUT /api/productos/:id</code> exitoso (el producto actualizado). |
| 201    | Created    | Se creó un recurso nuevo.                                           | <code>POST /api/productos</code> exitoso; la respuesta incluye el producto con su id autogenerated.                 |
| 204    | No Content | Éxito sin contenido que devolver.                                   | <code>DELETE /api/productos/:id</code> exitoso; la respuesta no tiene body.                                         |

| Código | Nombre      | Significado                                 | Cuándo se devuelve en este proyecto                                                                                                            |
|--------|-------------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 400    | Bad Request | La petición es inválida: error del cliente. | POST o PUT con datos que no superan <code>validarProducto</code> ; la respuesta incluye el array <code>errores</code> con todos los problemas. |
| 404    | Not Found   | El recurso pedido no existe.                | PUT o DELETE sobre un id inexistente, detectado porque <code>changes</code> fue 0.                                                             |

La elección deliberada del código correcto — 201 al crear, 204 al borrar, 400 ante datos inválidos, 404 ante recursos ausentes — es parte de lo que este proyecto enseña: los códigos comunican intención, no solo éxito o fracaso.

## 6. Evaluación formal de los objetivos pedagógicos

El proyecto se propone como material de aprendizaje integral. La siguiente tabla evalúa cada objetivo pedagógico, la manera en que el proyecto lo cumple y la evidencia concreta en el código.

| Objetivo pedagógico                                      | Cómo lo cumple el proyecto                                                                                                                                                                   | Evidencia en el código                                                                                                                                                                                                           |
|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Comprender el ciclo petición/respuesta HTTP              | Cada acción de la interfaz se convierte en una petición <code>fetch</code> y el servidor responde con código de estado y cuerpo; el navegador reacciona a la respuesta.                      | <code>public/index.html</code><br>( <code>guardarProducto</code> , <code>borrarProducto</code> , <code>cargarProductos</code> );<br><code>src/routes/productos.js</code><br>( <code>res.status</code> , <code>res.json</code> ). |
| Aplicar las cuatro operaciones CRUD de extremo a extremo | POST, GET, PUT y DELETE están implementados en el servidor, consumidos desde el navegador y verificados por pruebas de la cadena feliz.                                                      | <code>src/routes/productos.js</code> ;<br><code>public/index.html</code> ;<br><code>test/api.test.js</code> (casos 1 a 6).                                                                                                       |
| Separar presentación, lógica de negocio y persistencia   | La interfaz vive en <code>public/</code> , las reglas de negocio en <code>src/validators/</code> , la orquestación en <code>src/routes/</code> y la persistencia en <code>src/db.js</code> . | Estructura de carpetas de <code>src/</code> y <code>public/</code> ; <code>src/app.js</code> exporta la app sin escuchar.                                                                                                        |
| Validar la entrada en el servidor                        | <code>validarProducto</code> aplica reglas de negocio y acumula todos los errores; las rutas responden 400 con la lista completa.                                                            | <code>src/validators/productos.js</code> ;<br><code>test/api.test.js</code> (casos 7 a 12).                                                                                                                                      |

| Objetivo pedagógico                              | Cómo lo cumple el proyecto                                                                                                                                           | Evidencia en el código                                                                                                                                                                                                  |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Programar de forma defensiva                     | Los prepared statements impiden la inyección SQL y <code>textContent</code> impide la ejecución de HTML inyectado (XSS).                                             | <code>src/routes/productos.js</code> (consultas con <code>?</code> ); <code>public/index.html</code> ( <code>celda.textContent = valor</code> ).                                                                        |
| Utilizar asincronía con <code>async/await</code> | El arranque interactivo y todas las operaciones de red del frontend se secuencian con <code>async/await</code> , incluida la garantía de guardar antes de refrescar. | <code>src/server.js</code> ( <code>async function main</code> ); <code>public/index.html</code> (manejador del submit, <code>guardarProducto</code> , <code>cargarProductos</code> ).                                   |
| Practicar pruebas automatizadas                  | 13 pruebas ( <code>node:test</code> + <code>supertest</code> ) cubren el CRUD completo, los códigos de error y el seed, con aislamiento total de la base real.       | <code>test/api.test.js</code> ; <code>test/poblar.test.js</code> ; script <code>test</code> de <code>package.json</code> .                                                                                              |
| Modularizar el proyecto                          | Cada archivo de <code>src/</code> tiene una responsabilidad única y se comunica por los mecanismos de módulos de CommonJS.                                           | <code>src/app.js</code> , <code>src/server.js</code> , <code>src/db.js</code> , <code>src/routes/productos.js</code> , <code>src/validators/productos.js</code> ( <code>require</code> / <code>module.exports</code> ). |

La evaluación anterior permite concluir que el proyecto cumple de manera verificable los ocho objetivos planteados: cada objetivo cuenta con una realización concreta en el código y con evidencia comprobable en archivos y pruebas. zz-js01 se consolida así como un caso de estudio completo — del formulario al disco, y del disco de vuelta a la tabla — en el que el ciclo HTTP, el CRUD, la separación de capas, la validación, la programación defensiva, la asincronía, las pruebas y la modularidad no se describen en abstracto, sino que funcionan juntos en una aplicación real que puede ejecutarse, observarse y ponerse a prueba.